

Bài 1:

1. Giải thích lỗi: Trong vòng lặp `for`, giá trị của `leader[p]` bị lấy trực tiếp từ mảng. Nếu vòng lặp cập nhật chỉ số `p` trước khi duyệt đến một phần tử `i` khác (với `i > p`) cũng đang thuộc cùng tập hợp với `p`, thì điều kiện `leader[i] == leader[p]` sẽ bị sai do `leader[p]` đã mang giá trị mới. Phần tử `i` đó sẽ bị bỏ sót và không được cập nhật.

2. Testcase chứng minh:

`n = 3` (Mảng khởi tạo: `[0, 1, 2]`)

Chuỗi phép toán: `union(0, 1)`, sau đó là `union(0, 2)`

`i = 1, j = 2`

-Sau `union(0, 1)`: mảng trở thành `[1, 1, 2]`.

-Khi gọi `union(0, 2)` (với `p=0, q=2`):

Khi `i = 0`: Điều kiện `leader[0] == leader[0]` đúng. Mảng cập nhật `leader[0] = leader[2]`, trở thành `[2, 1, 2]`. (Lưu ý: `leader[0]` giờ là 2).

Khi `i = 1`: Điều kiện `leader[1] == leader[0]` tức là `1 == 2` (Sai!). Cập nhật bị bỏ qua. Mảng vẫn là `[2, 1, 2]`.

Kết quả: Dù 1 và 2 đã được gộp vào chung một tập hợp, nhưng `find(1)` trả về 1, trong khi `find(2)` trả về 2.

Bài 2:

Với chuỗi thao tác: `union(0, 1)`, `union(0, 2)`, ..., `union(0, n-1)`.

(a) Số thành phần liên thông : 1

Do phần tử 0 được nối với tất cả các phần tử còn lại, mọi phần tử đều cùng nằm trong một tập hợp duy nhất.

(b) Quick-find: Số lần cập nhật mảng : $n^2/2$

Ở mỗi bước `union(0, k)`, tập hợp chứa 0 đang có kích thước là `k` (gồm các phần tử 0, 1, ..., `k-1`). Vì cài đặt không thay đổi `leader[q]`, toàn bộ `k` phần tử này phải cập nhật ID sang `k`. Tổng số lần cập nhật là: $1 + 2 + 3 + \dots + (n-1) = n(n-1)/2$.

(c) Quick-union: Số lần truy cập mảng của `find(0)` : $O(n)$

Giải thích: Vì cài đặt không thay đổi `parent[q]`, mỗi phép `union(0, k)` sẽ biến đổi cấu trúc thành một cây dạng đường thẳng dài: `0->1->...->n-1`. Thao tác `find(0)` phải duyệt từ đáy (0) lên đến gốc (`n-1`), mất thời gian tỷ lệ thuận với `n`.

(d) Weighted Quick-union: Số lần truy cập mảng của `find(0)` : $O(1)$

Khi `union(0, 1)`, 2 cây có cùng kích thước (1), đề bài quy định thay đổi `parent[q]` nên phần tử 1 sẽ trở vào phần tử 0, khiến 0 trở thành gốc (kích thước 2). Ở các bước `union(0, k)` tiếp theo, cây của 0 luôn có kích thước lớn hơn cây của k (chỉ có cỡ 1), nên gốc của k sẽ luôn được trở trực tiếp vào 0. Cây tạo ra là một cấu trúc hình sao với 0 là gốc. Vì 0 tự là gốc của nó, `find(0)` trả về ngay lập tức với số lần truy cập mảng là hằng số.

Bài 3: COS226, Midterm f25, 2a

Ngoại trừ (0,4), (2,3), (5,7) thì còn lại đều có thể :

(0,4) : trước khi union, size của 2 set bằng nhau $\Rightarrow$ `parent(4)=0`. Theo như hình vẽ $\Rightarrow$ sai

(2,3) : trước khi union, size của set chứa 2 > size set chứa 3 $\Rightarrow$ `parent[3]` sau union phải bằng 4 $\Rightarrow$ sai
tương tự với (5,7)

Bài 4:

Có thể nhận 0,4,8

Ta thấy 3 nhóm : set1: {0,1,2,3} gốc là 0 ; set2: {4,5,6,7} gốc là 4 ; set3: {8,9} gốc là 8

Nếu không union set3 với bất cứ set nào thì `parent[8]=8`

Nếu union set3 với set1 $\Rightarrow$ `parent[8]=0`

Nếu union set3 với set2 $\Rightarrow$ `parent[8]=4`

Bài 5:

Hãy xét thời điểm ngay trước khi mũi tên **0 $\rightarrow$ 5** được tạo ra (tức là lệnh `union` giữa tập chứa 0 và tập chứa 5 được thực thi).

Dựa vào nguyên tắc số 2, toàn bộ 7 phần tử của nhánh gốc 0 bắt buộc phải được liên kết với 0 *từ trước khi* 0 trở vào 5. Nghĩa là lúc gộp, **kích thước cây gốc 0 đang là 7.**

Tổng số phần tử là 10. Trừ đi 7 phần tử thuộc cây gốc 0, tập hợp chứa gốc 5 lúc đó có kích thước lớn nhất cũng chỉ là **3 phần tử** (gồm 5, 7, 9).

Luật của WQU bắt buộc: **Gốc của cây nhỏ hơn phải trở vào gốc của cây lớn hơn.**

Vì kích thước cây gốc 5 (size = 3) < kích thước cây gốc 0 (size = 7), đáng lẽ ra **gốc 5 phải trở vào 0** (`parent[5] = 0`).

Tuy nhiên, mảng của đề bài lại cho thấy **0 trở vào 5** (`parent[0] = 5`).